

Meidan: Interaktiver Teddybär als Baby-Wein-Alarm

Prototyp für Interaktive Textilien

Projektvorstellung

ESP32 Go-Server Gemini Flutter-App

16. Mai 2026

Ausgangsidee

Ziel

Ein Kuschelbär wird zu einem unauffälligen, interaktiven Textil: Er erkennt laute Geräusche oder mögliches Babyweinen und informiert die Eltern über eine App.

- Elektronik verschwindet im Bär, das Objekt bleibt vertraut.
- Sensorwerte werden regelmäßig an einen Server gesendet.
- Der Server bewertet Messwerte lokal oder optional mit Gemini.
- Die Flutter-App zeigt Status, Pegel und Warnungen.



ESP32
Mikrofon

Technik im Textil
statt sichtbar am Gerät

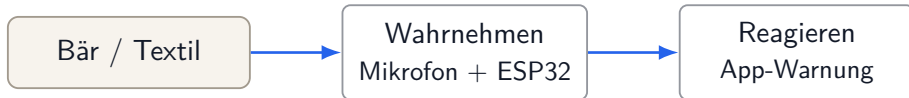
Warum Interaktive Textilien?

Textiles Objekt

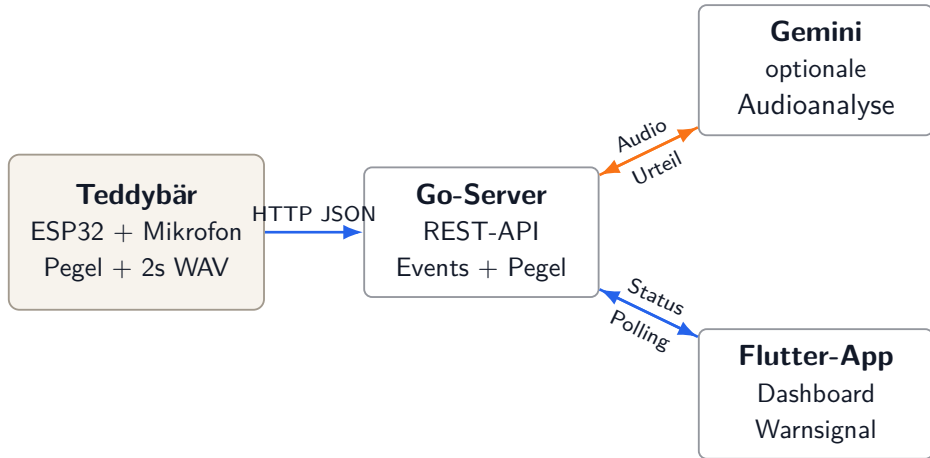
- weich, vertraut und kindgerecht
- Sensorik kann verdeckt integriert werden
- keine technische Hürde für die Nutzung

Interaktion

- Umgebung wird über Geräusche wahrgenommen
- Server erzeugt aus Messwerten ein Ereignis
- App übersetzt Ereignisse in verständliche Warnungen



Systemüberblick



Der Server ist die zentrale Stelle: Er sammelt Messwerte, bewertet Ereignisse und stellt den aktuellen Zustand für die App bereit.

Hardware im Bär

Bauteile

- ESP32 als Mikrocontroller mit WLAN
- analoges Mikrofonmodul, z. B. KY-037/KY-038
- Stromversorgung für den mobilen Prototyp
- weiche Platzierung im Inneren des Bären

Wichtige Parameter

Messfenster	120 ms
Pegel	max - min
Audio	2 s, 8 kHz WAV
Pegel-Upload	alle 3 s
Event-Upload	bei Weinen / alle 10 s

Im Sketch wird der analoge Eingang `SOUND_PIN = 34` verwendet. Die Empfindlichkeit wird über `CRY_THRESHOLD` und das Potentiometer des Mikrofons abgestimmt.

ESP32: Messlogik

- Der ESP32 liest den analogen Mikrofonwert in kurzen Fenstern.
- Der Pegel ist die Differenz zwischen höchstem und niedrigstem Wert.
- Erst nach mehreren Treffern wird ein mögliches Weinen gemeldet.
- Optional wird eine kurze WAV-Aufnahme als Base64 mitgesendet.

```
SoundReading reading = readSound();  
bool loudEnough =  
    reading.level >= CRY_THRESHOLD;  
  
if (loudEnough) {  
    cryHitCount++;  
} else {  
    cryHitCount = 0;  
}  
  
bool crying =  
    cryHitCount >= CRY_HITS_REQUIRED;
```

HTTP-Daten

Events

POST /api/events

```
{
  "deviceId": "esp32-1",
  "crying": true,
  "volume": 86,
  "confidence": 0.82,
  "mimeType": "audio/wav",
  "audioBase64": "...
}
```

Lautstärke

POST /api/volume

```
{
  "deviceId": "esp32-1",
  "volume": 1400
}
```

Die App liest später über GET /api/events und GET /api/volume die letzten Meldungen.

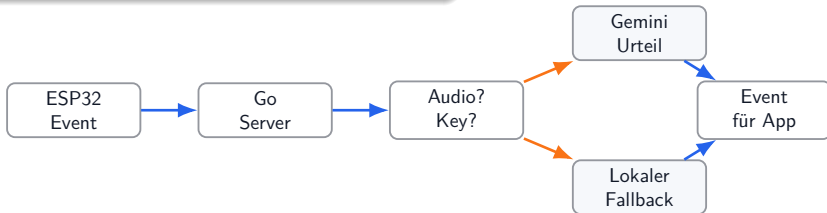
Server und KI-Bewertung

Aufgaben des Go-Servers

- REST-Endpunkte für ESP32 und App
- Speicherung der letzten Events und Pegel im Arbeitsspeicher
- CORS für Flutter-Web
- Server-Sent Events vorbereitet über `/api/events/stream`

Gemini optional

- Audio wird als audio/wav übertragen.
- Gemini liefert `crying`, `confidence`, `message`.
- Ohne API-Key oder bei Fehlern nutzt der Server lokale Regeln.



Flutter-App für Eltern

Dashboard

- Status: Bereit, Alles ruhig, Weinen erkannt, Sehr laut
- aktueller Lautstärkepegel
- Liste der letzten Ereignisse
- Server-URL kann in der App gesetzt werden

Benachrichtigung

- App fragt Server alle 3 Sekunden ab.
- Neue Warnungen können einen Warnton auslösen.
- Testbuttons erlauben Demo ohne ESP32.

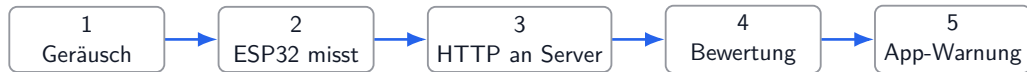
Meidan

Weinen erkannt

KI-Audioanalyse angefragt

Pegel: 1400

Datenfluss in der Demo



Normaler Ablauf

- Mikrofon erkennt niedrigen Pegel.
- Server speichert ruhige Meldung.
- App zeigt „Alles ruhig“.

Warnfall

- Pegel überschreitet Schwellwert oder Audio wird als Weinen bewertet.
- Server erzeugt Event vom Typ cry oder loud.
- App zeigt Warnung und kann Ton abspielen.

Demo-Plan

- 1 Go-Server starten.
- 2 Flutter-Web-App starten.
- 3 ESP32 im Bär einschalten.
- 4 Ruhigen Zustand zeigen.
- 5 Geräusch / Testevent auslösen.
- 6 Warnung in der App erklären.

Für eine Demo ohne Hardware können die Testbuttons in der App oder curl-Requests aus der README verwendet werden.

```
cd server
go run .

cd app
flutter run -d web-server \
  --web-port=8081 \
  --dart-define=SERVER_URL=http://localhost:8080
```

Grenzen des Prototyps

- **Audioqualität:** Ein einfaches Mikrofon im Stoff kann gedämpft oder verrauscht sein.
- **Kalibrierung:** Schwellwerte müssen an Raum, Mikrofon und Platzierung angepasst werden.
- **Datenschutz:** Audioanalyse sollte bewusst aktiviert, erklärt und abgesichert werden.
- **Energie:** Für einen echten Bär-Prototyp braucht es ein sicheres Akkukonzept.
- **Robustheit:** WLAN-Ausfälle und Serverfehler müssen für den Alltag abgefangen werden.

Nächste Schritte

Gehäuse und Kabelführung im Textil verbessern, echte Push-Benachrichtigungen ergänzen und Messdaten mit realistischen Geräuschsituationen testen.

Was gezeigt wird

- Ein Alltagsobjekt wird durch Sensorik interaktiv.
- ESP32, Server und App bilden eine vollständige Kette.
- KI kann die lokale Messlogik ergänzen, ist aber nicht zwingend nötig.

Kernaussage

Interaktive Textilien müssen nicht sichtbar technisch wirken: Der Bär bleibt ein weiches Objekt, reagiert aber über die digitale Infrastruktur auf seine Umgebung.

Backup: API-Endpunkte

Methode	Pfad	Zweck
GET	/health	Serverstatus
GET	/api/events	letzte Events
POST	/api/events	Weinen-/Ruhe-Event anlegen
GET	/api/events/stream	Event-Stream
GET	/api/volume	letzte Pegelwerte
POST	/api/volume	Pegelwert anlegen